

Introducing knowledge redundancy practice in software development: Experiences with job rotation in support work

Tor Erlend Fægri^{a,b,*}, Tore Dybå^a, Torgeir Dingsøyr^{a,b}

^a SINTEF Information and Communication Technology, SP Andersens vei 15b, 7465 Trondheim, Norway

^b Norwegian University of Science and Technology, Department of Computer and Information Science, Sem Sælandsvei 7-9, 7491 Trondheim, Norway

ARTICLE INFO

Article history:

Received 12 October 2009

Received in revised form 10 May 2010

Accepted 7 June 2010

Available online 11 June 2010

Keywords:

Job rotation

Personnel rotation

Empirical software engineering

Agile organizations

Organizational learning

Software development

ABSTRACT

Context: Job rotation is a widely known approach to increase knowledge redundancy but empirical evidence regarding introduction and adoption in software development is scant. A lack of knowledge redundancy is a limiting factor for collaboration, flexibility, and coordination within teams and within the organization.

Objective: The scientific objective of this investigation was to explore benefits and challenges with improving knowledge redundancy among developers participating in job rotation. There were two practical objectives; (a) to establish customer support as a legitimate organizational function that would shield developers from support enquiries, and (b) to contribute to improved flexibility in project staffing by enabling overlapping product experience among developers.

Method: We used action research to integrate organizational change with scientific inquiry. During a period of eighteen weeks, nine developers rotated to customer support. We collected data throughout the period of collaboration; in meetings, from comprehensive interviews, and from customer support work logs.

Results: Perceptions of reduced efficiency and unnecessary redundancy outweighed benefits of shielding and learning about different products. Although there were strong indications of increased knowledge redundancy, the benefits were not sufficient to justify job rotation. Job rotation was abandoned after the trial period.

Conclusions: Job rotation can contribute to improved knowledge redundancy. Benefits of knowledge redundancy include innovation stemming from integration of different knowledge domains and improved appreciation of organizational concerns. However, knowledge redundancy incurs a collective cost that must be amortized and legitimized by the organization. An adoption process that accommodates open and trustful discussion among all involved stakeholders is therefore encouraged.

© 2010 Elsevier B.V. All rights reserved.

1. Introduction

Software development is collaborative, non-routine, knowledge intensive work in which the ability to combine and integrate knowledge from a multitude of sources is critically important [71,75]. Although software development can contain elements of routine work and repetitiveness, it is mostly concerned with creative problem solving [1,11], even during maintenance and support [35]. Its intellectual basis in problem solving implies that software development is fundamentally a process of organized, collaborative learning [24]. Additionally, many software organizations face competitive and turbulent environments that increase the impor-

tance of being able to benefit from the employees' range of skills and competences [36,54]. By utilizing its access to cognitive diversity among specialized individuals, the software organization can improve its potential for learning and innovation. For groups and the organization, a level of overlap, or redundancy in knowledge is paramount for effective collaboration and coordinated action [40]. Specialized individuals can also benefit from acquiring knowledge from other domains; a diverse repertoire of knowledge contributes to integrative learning and improved problem solving capacity [44]. Hence, the software organization should strive to accommodate individuals' need for diverse knowledge and the collective's need for knowledge redundancy.

Job rotation is a practice that gives individuals direct experience with work pertaining to different knowledge domains and therefore contributes to knowledge redundancy [54]. By legitimizing peoples' movement among already existing groups within an organization, job rotation can contribute to overcome barriers enacted

* Corresponding author at: SINTEF Information and Communication Technology, SP Andersens vei 15b, 7465 Trondheim, Norway. Tel.: +47 73 59 30 07; fax: +47 73 59 29 77.

E-mail addresses: Tor.E.Fegri@sintef.no (T.E. Fægri), Tore.Dyba@sintef.no (T. Dybå), Torgeir.Dingsoyr@sintef.no (T. Dingsøyr).

by departments, projects, or product groups. Such organizational routines can be effective at integrating knowledge between different domains [36]. The literature discusses numerous benefits of job rotation such as improving knowledge redundancy and organizational knowledge creation [54], widening the awareness of the organization's range of activities [57], and bridging of individuals' thought worlds [34].

However, the literature gives scant attention to actual adoption. Establishing and sustaining new organizational learning practices can be highly challenging, for example due to confrontations with established values and norms [5]. Hence, it is relevant to also investigate challenges related to the introduction of job rotation. Furthermore, most empirical research on job rotation has been conducted in manufacturing organizations. Because of the particularly important role of collaborative learning in software development it is essential to better understand how practices for knowledge redundancy can be applied in software development. From a practical perspective, job rotation is an attractive practice because it legitimizes direct practical experience with a diverse set of knowledge domains for the participants. Direct experience allows the individual to explore both practical and intellectual dimensions of the work and hence improve the efficiency of learning through active experimentation and reflection [44]. Besides, direct experience can reveal a richer set of thoughts about actions [9], which is beneficial for gaining deeper insights about phenomena.

The action research reported here studied a trial of job rotation at a Norwegian software organization that we will call MapIT (a pseudonym). Leading up to the trial was a perceived problem within MapIT that developers became excessively disturbed by frequent customer support enquiries. The enquiries appeared randomly and often demanded immediate attention by a developer. Developers referred to this phenomenon as 'the quagmire' to represent their perception of being personally responsible for software systems throughout the system's lifecycle [50]. MapIT possessed a wide product portfolio resulting from numerous, small projects; development experience with any particular customer enquiry was typically restricted to a named individual. Hence, it was often difficult for developers to reject this responsibility.

There were two practical objectives with the trial: The first was to establish a more formalized support service that would provide developers with some shielding from support enquiries, while the second was to contribute to expand developers' knowledge on the range of MapIT's products. The main scientific objective was to contribute to improved understanding of knowledge redundancy in software development; in particular challenges regarding introduction and institutionalization of knowledge redundancy practices. The research question we wanted to answer was:

What benefits and challenges can arise from introducing knowledge redundancy interventions based on job rotation in software development?

The main contribution of this article is an analysis of the key benefits and barriers to sustained use of job rotation in MapIT combined with a discussion of theoretical and practical implications. Software organizations that are considering the use of job rotation for improving knowledge redundancy may benefit from evaluating our findings and reflecting upon the similarities of the contextual features of their study with their local context.

The remainder of this article is organized as follows. Section 2 presents theoretical background. Section 3 explains the research approach. Section 4 presents the action research stages: diagnosing, action planning, action taking, and evaluation, while Section 5 specifies the learning outcomes of the action research along with implications and limitations of our findings. Section 6 concludes the article.

2. Theoretical background

This section gives the theoretical background to the investigation. First we clarify what we mean by knowledge. Subsequently, we explain the critical role of knowledge redundancy in software development practice. Next, we introduce implications of organizational issues pertaining to knowledge redundancy in software development. Finally, we summarize existing research on job rotation and position our own research within this body of knowledge.

2.1. Knowledge and knowledge redundancy

Knowledge can be characterized along a continuum between explicit or tacit. Explicit knowledge is knowledge that is codified or articulated; for example as concepts explained in documents or spoken narratives being shared among a group of people [12]. We can think of explicit knowledge as information [2]. Tacit knowledge, on the other hand, is in the realms of peoples' cognitive faculties; it is held by individuals as concepts and skills, or shared within groups as socially enacted genres and 'social knowledge' [2,17,56]. Tacit knowledge is silent, i.e., it is not articulated but rather taken for granted and is ultimately intertwined in action. Henceforth, we refer to this end of the continuum as tacit knowing in accordance with Polanyi [60]. Both explicit knowledge and tacit knowing are fundamental in software development just as practical and intellectual knowledge are mutually indispensable for working professionals. Similarly, both group and individual knowledge does epistemic work that the other cannot [17].

Knowledge that is shared among a group of individuals represents a level of *knowledge redundancy* or overlap in individuals' knowledge [62]. According to Piaget, "new knowledge always depends on existing knowledge" [77] and social interaction is shaped by already shared knowledge [55]. Correspondingly, knowledge redundancy creates common references for people's creation of new knowledge [64]. Further, as the level of redundancy increases within the group, individuals find it easier to share new knowledge and organizations are able to coordinate their work more effectively [40]. Because knowledge is partly justified individually, there will necessarily be variation in people's interpretation of the same concepts. Knowledge is also partly justified by the group or forces in the individual's environment. The dominance of group norms on justification depend on the autonomy of the group [7] and the level of collaboration; for closely collaborating project teams the group's norms of justification are likely to dominate individual justification and will therefore contribute to isomorphic understanding [12,70]. Similarly, infrequent collaboration or departmentalization contributes to diversity in people's interpretation of the same concepts [13,14]. Although diversity of interpretation may be significant, knowledge will always have collective aspects because language itself is social knowledge [70]. Hence, knowledge redundancy is mediated by the nature of collective action.

2.2. Knowledge and knowledge redundancy in software development

Knowledge is of particular interest to our discussion of software. Software is codified knowledge [63] that is developed and evolved in controlled, social collaboration among a multitude of stakeholders such as software practitioners, customers, and users [32]. A fundamental property of software is its non-physical form; software code is essentially a large set of abstract instructions possessing unlimited complexity, flexibility, and revisability. Software exhibits non-linear behavior and do not conform to laws of nature. One consequence is that it is inherently hard to build models of software that allow accurate reasoning about the system's qualities [11]. For all but the simplest of software systems the evaluation

can only be performed during use [28]. Furthermore, in the turbulent environments facing many software organizations, documentation is often considered less valuable than the pursuit of software features [63]. The combination of little relevance of attention to documentation and the difficulty of building models supporting qualitative reasoning mean that the software code itself becomes the primary object for understanding its behavior.

Software systems are social constructions whose meaning is continuously defined and re-defined by the context of use [55,59]. Software practitioner's use of the software system is primarily characterized as designing, constructing, or maintaining it, the software system under development represents an actual artifact [43] whose meaning is enacted through the complex network of design decisions that constitute its development history [28]. Collaboration in software development intertwines learning with work [12] and the social interaction stimulates the creation of new redundant knowledge that benefits the group [55]. The collaboration results in knowledge redundancy in the form of group agreements and design decisions and shared explicit knowledge in the form of narratives for problem solving and software artifacts such as designs and code [75]. Furthermore, as codified knowledge, the software artifacts serves as objects supporting external justification towards other stakeholders [70]. Hence, there are inherent properties of software development that contributes to knowledge redundancy.

However, the factors contributing to knowledge redundancy are largely counterbalanced by the integral role played by individuals' tacit knowing and intuitions. Software professionals depend largely on privately held assumptions, rationales, and decisions when developing software systems [63]. Despite collaboration, formation of diverse knowledge and skills occurs naturally within groups of people. Individuals have personal experiences that contribute to their accumulated knowledge and specialization [44]. Furthermore, specialization is associated with high levels of proficiency [44] and the ability to solve more complex problems [29], which are important motivations for people to specialize rather than seek overlapping knowledge. Further, given the high complexity of software systems, people are naturally led to focus their attention to particular modules, technologies, or aspects of the software system being developed, thereby promoting acquisition of specialist knowledge. Indeed, principles for 'sound' software development such as modularization, low coupling, encapsulation, abstraction, and even the divide-to-conquer strategy to problem-solving seeks to accommodate people's limited cognitive capacity by encouraging a focusing of attention to certain aspects at a time [16,63]. Thus, specialization of knowledge is a normal property of software development.

A consequence of knowledge specialization in software development is that program comprehension constitute up to 50% of maintenance work [18]. Some even argue that a full understanding of the software requires experience with developing it [43]. It appears as if the software code becomes an integrated part of the developer's personal practice [60] and the enabling of knowledge redundancy is complicated.

2.3. Organizational barriers to knowledge redundancy

Organizations operating in competitive environments must constantly seek to improve efficiency and resource utilization. A traditional approach in many areas of technology development is to seek gains in efficiency from standardization and routinization. This manufacturing view of work organization is based in an assumption that work is predictable and largely repeatable; the organization becomes 'a machine' where skills are sought implemented in carefully planned routines [52]. A manufacturing view implies a preference for standardization of processes, hierarchical

structuring of information flow, and fixed division of labor. In this view knowledge redundancy becomes an expense [8]. However, software development is fundamentally incongruent with the manufacturing view of work organization. Software development is non-linear, continuous knowledge creation [53,75]. Software development bears stronger resemblance to experimentation [28], improvisation [23], and creative problem solving than to manufacturing [1]. Furthermore, the non-clerical, non-routine character of work is significant not only in the actual coding; but also in the work of establishing requirements and product evolution – such as maintenance and support [35,65,74]. In such contexts, the fixed hierarchical organizational structures will inhibit, rather than support the need to dynamically, fluently, and effectively integrate knowledge from the organization's members [36]. And so, nurturing the software organization's knowledge creation capability is likely to be more important than stimulating predictability and standardization.

As a response to the lack of flexibility in bureaucratic organization of work many software organizations use projects as ways of structuring software development [21]. Projects transfer more control and autonomy to the group level and hence the project based organization represents a step away from the bureaucratic organization [39]. Control and autonomy is transferred to the project by assigning to it defined goals, a set of associated software practitioners and a time limit for completion. The collaboration becomes a socially enacted structure guiding the construction of local language, local substantive rationality, local identity, and project-specific 'thought worlds' [7,12,33] that make projects stimulating and effective learning environments for software practitioners [6]. The intensity of learning and the direct experience with the actual software system implies that the software system itself and the project participants will constitute the organization's best knowledge of the software system.

Furthermore, projects may easily enact even higher barriers to knowledge integration than the bureaucratic manufacturing organization [39]. Although knowledge redundancy among the project members is supported by the project form [21], the project group has an inherent tendency to construct and evolve strong barriers towards its environment [15] making knowledge sharing between projects difficult. A software organization may have legacies of innumerable projects. Small projects, wide product portfolios and low management attention to knowledge sharing contribute to a proliferation of multiple 'knowledge silos.' Not only may these silos inhibit critical information exchange during product development [40] and contribute to the shaping of organizational structures [55], but the knowledge clustering constitutes fundamental, cementing blueprints for the future technologies produced by the organization [15]. Considering the turbulent environment in which many software organizations operate, the project-based approach to software development can easily result in multiple islands of non-sharable knowledge and create substantial barriers towards innovation [36,54]. Furthermore, while physical artifacts can serve as effective boundary objects between groups in many disciplines, the intangible nature of software is a complication [26]. Hence, knowledge is inherently locked into the software system and direct experience is necessary to unlock it. Therefore, beyond the project level, knowledge creation is a major dilemma in software development.

Key enabling factors for knowledge creation is knowledge sharing and integration [36,54]. Research in organizational learning has emphasized the value of practice; people acquire and share knowledge in socially situated work. Learning in the organization occurs in the interplay between tacit and explicit knowledge while it crosses boundaries of groups, departments, and organizations as people participate in work [17,54]. The process should be situated in shared practice with a joint, collective purpose [12,14,15]. But it

can be difficult to establish such practices in software development without auxiliary assistance [31]. The responsibility, then, resides with the organization to accommodate such practice.

2.4. Job rotation

One practice to increase knowledge redundancy is job rotation. Job rotation allows experience from different roles, tasks, and domains within the organization and hence increases the overlap of knowledge among the organization's members [20,54]. While specialization is a natural tendency for many individuals, job rotation seeks to counter this tendency by legitimizing individuals to experience a wider range of tasks [57]. Job rotation is not confined to particular areas of work. Job rotation therefore has the potential to bypass project-level barriers to knowledge redundancy.

The literature discusses several organizational benefits of job rotation. It is claimed to be a characteristic of learning organizations that may contribute to higher innovative performance [46], and be beneficial to tacit knowledge sharing [68] and knowledge creation [66] through development of knowledge redundancy [54]. Job rotation can improve motivation and reduce fatigue [45], team efficiency [38], problem-solving skills [3], and can enable people to understand larger aspects of an organization's processes [20]. Job rotation also allows management to appreciate people's range of competencies [27]. Un [72] theorizes that job rotation increases psychological safety among participants and suggests that it creates a better environment for exploration of new knowledge. Exploration of new knowledge is often attributed to the interaction between knowledge from different domains [33,54] but is often inhibited because knowledge is embedded in local practices and artifacts [14]. For software development, as we have argued, knowledge is embedded in the software systems and direct experience is needed to understand them. Personal experience, such as enabled by job rotation, can improve the ability to create shared understanding.

Job rotation also fosters improved diversity in individuals' knowledge that can improve their problem-solving and creative performance [54,69]. Furthermore, job rotation can contribute to improved efficiency in cross-functional teams. A problem in groups is that people tend to prefer to talk about already shared knowledge which make it difficult in cross-functional teams to benefit from the variety of knowledge available to the group [34]. Job rotation can hence increase the necessary redundancy of knowledge required for cross-functional teams to work efficiently. Most of the identified research on job rotation has focused on productivity effects rather than adoption challenges and peoples' experience. Considering the difficulty of implementing sustainable organizational learning programs [5] this topic is of high importance.

Further, empirical research on job rotation in software development is scant. Wang [76] observed that job rotation to customer support contributed to developers' motivation while providing a valuable source of customer requirements (pp. 155, 165). Fægri [30] argued that job rotation to customer support improved knowledge diversity but created conflicts with project-based efficiency norms. Arent et al. [4] state that job rotation is an example of an exploratory type of organizational learning strategy which will benefit tacit knowledge creation, but without any empirical basis. Similarly, without empirical basis, von Krogh et al. [73] suggest the use of job rotation to "broaden the understanding of a project, increasing sensitivity to coupling of tasks, and improve management of interfaces."

Further scientific inquiry dealing with the actual adoption of job rotation and related benefits and challenges in software development practice can therefore give a valuable supplement to our current understanding.

3. Research methodology

We used action research as our methodological framework. This section justifies our choice of action research, the case selection, our data sources, and fundamental aspects of our data analysis.

3.1. Justification of action research

Action research was an appropriate research methodology for this investigation for several reasons. First, the study's combination of scientific and practical objectives is a good match with the basic tenet of action research, which is to merge theory and practice by commending that real-world problems are solved by theoretically informed actions in collaboration between researchers and practitioners [37].

Furthermore, knowledge redundancy from job rotation is a complex phenomenon due to its roots in social practice (cf. Section 2). Action research is particularly aimed at investigations of such complex social phenomena [10]. Action research embraces the complexity of human action by accepting that the social setting will shape concepts, truth, rationality, and the practical action itself [9]. Additionally, little research has yet been done on job rotation in software development, which suggests that theory is still in a nascent state. This gives strong motivation for using a flexible research approach such as action research [25,42].

3.2. Action research adoption

In order to ensure both rigor and relevance in the research, we adopted the five principles of canonical action research (CAR) described by Davison et al. [22]. Table 1 explains how each principle is addressed.

3.3. Case selection

We had three reasons for conducting this action research at MapIT. Firstly, we believe that MapIT represents a typical, and

Table 1
Adoption of CAR principles.

1. *Researcher-client agreement.* We have a long history of research collaboration with MapIT focusing on software process improvement. The study presented here was conducted as part of an on-going action research project in which MapIT and the researchers participate together with a number of other software organizations. This particular investigation was defined as a trial of job rotation to customer support. MapIT's only requirement towards the researchers was that we agreed to complete and present a thorough evaluation at the end of the trial.
2. *Cyclical process model.* We adopted the five-stage, cyclical process model proposed by Susman and Evered [67]: Diagnosing, action planning, action taking, evaluating, and specified learning. One full cycle was completed after which MapIT decided to abandon job rotation. Section 4 elaborates on each stage.
3. *Theory principle.* Our main theoretical ambition was to expand current understanding of improving knowledge redundancy in software development using job rotation. In particular, we wanted to investigate how job rotation could enable knowledge sharing across technology and product boundaries. Theoretical framing is finalized during diagnosis [22] which we present in Section 4.1.
4. *Change through action.* We supported MapIT in initiating and planning of the trial. Our engagement during the trial was moderate, similar to the facilitative researcher engagement [10] and participant action research [67]. MapIT members involved in customer support activities during the trial were asked to register data concerning this in a support log.
5. *Learning through reflection.* We accommodated extensive reflective learning by organizing interviews, review meetings, and personal meetings after then trial. We subsequently presented our intermediate evaluation results [30]. Valuable insights emerged throughout these meetings which enabled us to build a more comprehensive picture of the change process including organizational constraints and idiosyncrasies [22].

possibly a *paradigmatic case* that exemplifies and illustrates general challenges of importance for other software organizations [29]. Of particular interest was MapIT's challenge related to balancing product-embedded specialist knowledge and knowledge redundancy. Due in part to a history of mergers and acquisitions, MapIT had accumulated its customers from different domains. As a result, MapIT offerings consisted of a widely heterogeneous portfolio of products and services that included electronic document management systems, geographical information systems, and related services such as consulting and service hosting. Prior research [51], and our own diagnosing (cf. Section 4.1), convinced us that there were significant problems related to a lack of knowledge redundancy. In particular, the seemingly impenetrable knowledge barriers between projects seemed to match existing theory regarding knowledge redundancy in software development (cf. Section 2.3). This would imply that findings could provide useful insights for other organizations.

Secondly, action research rests fundamentally upon a solid researcher-client infrastructure [22]. Our ongoing constructive collaboration with MapIT gave us the necessary confidence that this project could be realistically completed and evaluated. Among the roughly 30 employees at MapIT we had talked with most of them before and some of them at numerous occasions.

Thirdly, we had reasons to believe that the lack of knowledge redundancy, if improved, would give MapIT benefits in the near future. The company had experienced a steady growth in sales; with a surge in the demand for integrated solutions. This included integration of MapIT's own offerings as well as integration of MapIT offerings with other vendors' products. Knowledge redundancy can benefit organizations' ability to integrate competence and skills across domains [36]. Hence, improvements in knowledge redundancy had the potential of benefitting MapIT's capabilities in meeting future demands and MapIT therefore was a relevant organization for our action research [67].

3.4. Data sources

We used four types of data sources as the empirical basis for our investigation: meeting notes, semi-structured interviews, support work logs, and existing research publications reporting from prior collaboration with MapIT.

Table 2 gives a summary of our data sources. In addition, numerous visits to MapIT gave us many possibilities to talk informally with participants and management before, during and after the trial. Such opportunities occurred frequently by the coffee machine or around the lounge area. These talks allowed us to build a richer picture of MapIT and the trial.

We immediately reviewed and transcribed interviews and meeting notes to improve accuracy.

Throughout the article we will occasionally refer to individuals at MapIT that we interviewed or with whom we had meetings. Table 3 summarizes these individuals and lists their roles in the organization. To simplify the presentation we list a role indicator in parenthesis after each name.

Table 2
Data sources.

Source type	Duration of event	Volume
Meeting notes	1–4 h	Eight meetings (four diagnosing/action planning meetings, one review meeting, two individual meetings, one meeting presenting scientific results [30] for MapIT)
Semi-structured interviews	15–60 min (avg. 28 min for developers, avg. 21 min for others)	16 interviews (totaling 77 pages transcribed)
Support work logs	18 weeks	196 forms filled in during the whole duration of the trial. The logs included a total of 2577 support work events lasting 647 h
Existing research at MapIT	N/A	Three research papers [49–51]

Table 3
Subjects and roles.

Role	Name (role indicator)
Developers, Java group	Andrew (J), David (J), Stuart (J), Teresa (J)
Developers, .NET group	Chris (N), Eric (N), Gerry (N), Michael (N), Matthew (N)
Developer, both Java and .NET	Simon (J & N)
Customer support 1st line, marketing	Carl (S)
Consultant	Ken (C)
Development managers (2), product manager, consultant manager, general manager (respectively)	Mark (M), Keith (M), Anthony (M), Peter (M), Nick (M)

Our interviews have 100% coverage of participants in the rotation trial (nine participants). We also interviewed seven other subjects that we believed had relevant roles to the investigation including Matthew (N) which was a .NET developer that did not participate in the rotation due to responsibility as team-manager.

3.5. Data analysis

Meeting notes and interviews were coded in a tool for qualitative data analysis (NVivo). Data analysis was driven by grounded theory [19]. However, we did not start without any knowledge of MapIT. Rather, we benefitted from prior research at MapIT in building an understanding of the organizational and technological context [49–51]. This is important because the value of qualitative research depends on rich and elaborate contextual descriptions [29].

Furthermore, in contrast to passive qualitative research where the researcher must remain open for a vast range of interpretational frames, the action itself is the primary origin of interpretation in action research [10]. Based in the event of introducing job rotation, our scientific objective – and our origin of interpretation – was to understand how developers perceived the new practice of gaining overlapping knowledge from working on other developers' products. Also, because knowledge redundancy is a collective phenomenon pertaining to overlap of knowledge within a group, it can be difficult for individuals without management roles to recognize knowledge redundancy effects. Therefore, in our analysis we also wanted to understand how job rotation, as an organizational practice for knowledge redundancy, was considered from both an individual and a collective (organizational) perspective.

We started out with an initial phase of extensive and highly detailed open coding of our data. The result of the open coding process was a set of 283 low-level concepts describing potentially relevant opinions and statements. Following this, we started an axial coding process whereby we grouped these concepts into higher-level categories by looking for cues that would bind concepts together. As new insights emerged also after the open coding, for example from new meetings, reflection or discussions, we found great use of the tool as it easily accommodated constant comparisons between the transcribed data with the gradually emerging coding structure.

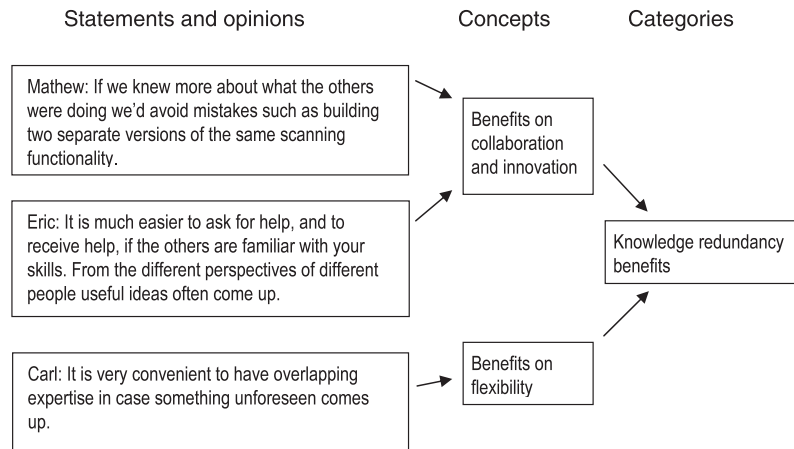


Fig. 1. Example coding progression.

Five core categories arose from our analysis: conceptualization, contextual factors, factors pertaining facilitation of knowledge redundancy, benefits of knowledge redundancy, and barriers to the development of knowledge redundancy. Fig. 1 shows an example progression from raw data (three transcripts from the interviews) via two concepts (“benefits on collaboration and innovation” and “benefits on flexibility”), to the final core category (“knowledge redundancy benefits”).

Furthermore, our coding tool had functionality for sophisticated queries, which supported further exploration and investigation of relationships in our data. For example, we were able to supplement interpretation of individual's statements with cross-group comparisons using such queries.

4. Action research

We now describe the unfolding of the action research phases structured according to the five phases of action research prescribed by Susman and Evered [67]: diagnosing, action planning, action taking, evaluating, and specifying learning (Section 5). The key findings for each phase are summarized in Table 4.

4.1. Diagnosing

In the period February to April 2008 we participated in four meetings with developers and managers at MapIT. The objective was to understand more of the quagmire problem and challenges of organizing the support work. We also had the benefit of extensive insight into software development at MapIT prior to these meetings from a history of research collaborations. The challenge with lacking knowledge redundancy among developers was noted

in prior investigations [49,50]. Prior collaboration also contributed to invaluable confidence between the researchers and people in MapIT.

In 2008 MapIT employed roughly 30 people of which 16 were software developers. The developers were seated in an open-plan area with circular four-person desks. People with other functions were located in normal offices nearby the open-plan area (see Fig. 2). There were also some spare offices available for special needs, such as developers doing pair programming and other meeting activities.

Development activities were predominantly organized in projects that historically had been small – staffed by 1–3 developers. Occasionally, larger projects were used – staffed by 4–6 developers. However, the dominance of small projects had created a strong prevalence of specialist knowledge which inhibited backup behavior in teams [49]. Another strong indicator was developers' perception of having no peers to offload work to that concerned prior projects. Developers referred to this problem as ‘the quagmire’ [50]. The name is symbolic; it illustrated developers' dismay of being held personally responsible for prior projects in addition to active projects. Related to the quagmire problem were difficulties of establishing shared leadership in the Scrum teams [51]. The organizational culture – mirrored by individuals' strong responsibility for prior projects – re-enforced specialization and ‘turf guarding.’

Both management and developers raised concerns about the unwieldy product portfolio. A developer mentioned that there were examples of products offering the same functionality but without reusing any code. We got no precise answers to questions on the number of products, but guesses fell within a 15–35 range. This wide range of estimates made us suspect that the awareness of the breadth of the product portfolio was poor.

Table 4
Action research phases and findings.

Phase/duration	Findings
Diagnosing (February 2008–April 2008)	Identified the quagmire problem and the related challenge of limited flexibility in staffing development projects. We framed the problem within organizational learning theory, and more specifically as a lack of knowledge redundancy
Action planning (March 2008–April 2008)	Discussed alternative actions; (1) all developers taking part in rotation, (2) small number of people taking part in rotation, (3) permanent 2nd line staff
Action taking (May 2008–August 2008)	Decided to implement alternative (1). The support room was configured and furnished, the rotation schedule was defined and job rotation was initiated
Evaluating (September 2008)	Interviewing participants and employees with related roles, analyzing interview transcripts and customer support work logs. Comparing results to objectives and expectations
Specifying learning (October 2008–September 2009)	We reported general findings to MapIT and initiated contribution to scientific community [30]. It was decided to abandon job rotation and appoint a permanent staff for 2nd line customer support. In September 2009 we presented the paper [30] in a meeting at MapIT. There were 12 participants

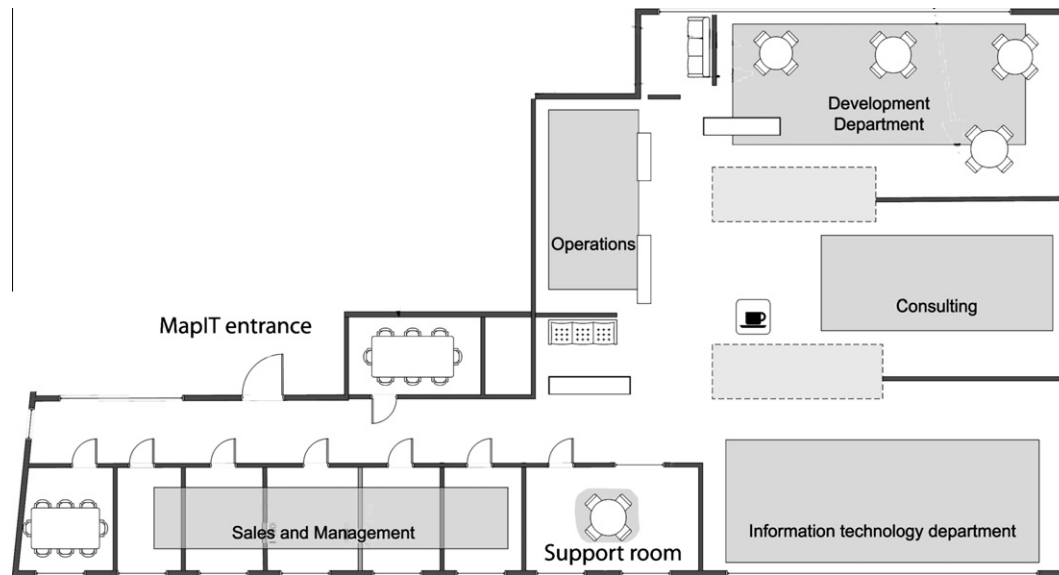


Fig. 2. Office layout.

Developers in MapIT were also specialized according to technology platform. Developers could be categorized according to their dominant programming environment; the Java group had five developers and the .NET group had 11 developers. Document management applications were mostly based on .NET while applications dealing with geographical models and maps were developed in Java. Although there were a few projects exploiting both technologies, the two groups were largely separate. The grouping was reflected by developers' group identity, such as a pronounced 'us and them' language.

Customer support is an integral part of technology development as it fundamentally connects the designers with the users. Because software systems are so flexible, the linking of designers and users fuels a vital, ongoing interpretation process subjecting the technology to emerging uses [55] and hence is an important enabler for maintaining the technology's momentum and commercial success [41]. In MapIT, customer support was tightly interwoven in developers' daily activities. However, supporting customers was associated with cumbersome and unplanned breaks in the flow of thoughts. Chris (N) explained: "Occasionally, there would be so many customer enquiries that I couldn't do any productive project work. It really chopped up my workflow." His statement illuminated a fundamental problem. Productivity was associated with working on active projects while supporting existing systems was not.

Developers' specialization on small products and a specific technology platform implied that it was difficult to share support work in MapIT. Furthermore, there was a general favoring of working code over documentation. Stuart (J) explained: "Because documentation is so modest, only developers that understand software code are able to deal with many of the enquiries." This was evident from existing practice. Carl (S), the only dedicated support staff, did not have skills in software development so he could only help customers with enquiries involving installation and infrastructure issues. For other enquiries he referred the customer to the developer he believed was most experienced with the software in question.

The practice of making referrals and assigning support tasks to individuals in MapIT both reflected and enacted the specialization of knowledge [58]. Also, there were concerns that products had poor quality and that bugs were not dealt with properly. Peter (M), the current consultant manager, warned that MapIT was "un-

able to learn from mistakes." Mark (M), the current development manager, commented: "We are too comfortable living with our mistakes." MapIT seemed to adhere to the picture of software organizations that had learned to fail [47].

Nick (M), the MapIT general manager, empathized with the interrupted developers and referred to the quagmire as "the cause of disruptions in the machine." Furthermore, in discussions regarding the challenge of innovation and business strategy he expressed similar concerns: "New products and technologies incur significant disturbances." He saw the proliferation of integrated solutions as an important trend for MapIT but noted: "You lose control when more actors are involved. And customer support is more demanding with systems that should interoperate." By these statements he created an image of MapIT as a machine concerned with efficient production of software and services rather than innovation, learning, and cooperation. But MapIT could hardly be seen as a machine. Applying the machine metaphor to MapIT [52] leaves the image of MapIT as consisting of a multitude of fairly short-lived 'project machines,' each with its own rationality; ensuring that the project produced a useful software system after which the project could be abandoned.

MapIT's turbulent business environment, the problems of lacking backup behavior, the difficulties of establishing shared leadership, the quagmire problem and the missing of obvious reuse opportunities among products suggested that MapIT's mechanisms for ensuring knowledge redundancy and knowledge integration were not effective. It appeared that each software system developed by MapIT constituted an island of knowledge and that little was done to accommodate the sharing of knowledge between them. Furthermore, developers did not experience support work as legitimate use of time.

Our theoretical framing was gradually evolving throughout the diagnosing phase. MapIT's learning disability combined with the multitude of knowledge silos constituted by each project seemed to confirm existing theory that unlocking knowledge embedded in software systems depends on direct and practical experience (cf. Section 2.3). Furthermore, we recognized the individual projects' local rationality as barriers to holistic, integrative learning practices spanning products and technology groups. Also, the identified need for increasingly integrated solutions among MapIT's customers gave reasons to investigate the value of cross-domain knowledge creation [54] and organizational routines that would

create opportunities for joint problem solving [12,15,36] and improved knowledge about the variety of products.

4.2. Action planning

The transition from diagnosing to action planning was gradual. In collaboration with MapIT, we discussed possible strategies for improving the situation. A short-term goal was to improve shielding of developers from interrupting support enquiries and while improving knowledge redundancy to alleviate the problem of low flexibility in task allocation. In the long term, MapIT would need to enable cross-product knowledge sharing to accommodate the challenge of innovation with integrated solutions. We emphasized the value of direct experience to understand software systems. However, based in prior research collaboration [51] and the ongoing discussions, we had a suspicion that MapIT management was primarily concerned with optimalization and bureaucratization and failed to appreciate the benefits of knowledge redundancy. We framed this challenge as an example of organizational learning [5] and explained the viability of action research to make it happen [10].

It was during these discussions that the idea of trying job rotation among developers to customer support came up. If developers rotated to a designated room for joint customer support work with Carl (S) the critical concerns might be solved. Primarily, the combined expertise of Carl (S) and a developer working as a team would ensure significant shielding. If they failed to resolve the enquiry it could be passed onto the product specialist. Secondly, we know that maintenance and support of software is highly demanding but gives extremely rich learning opportunities [74]. And because knowledge is inherently embedded in the actual software code, the rotation into the customer support role would contribute to a gradually increasing overlap in knowledge among the participants. This would subsequently improve developers' ability to work with a wider range of products. Besides, we hoped that it would help to improve their appreciation for the range of functionalities and features offered by different products, even between the .NET and Java groups, and catalyst ideas for future integration projects.

We discussed two distinct variants of job rotation in customer support. One was to engage all developers in rotation. The other was to engage 3–4 developers in rotation. MapIT had discussed a third alternative to remedy the problem prior to our collaboration. This alternative was to appoint a permanent staff to customer support. Thus, we had three distinct alternatives to improve the situation with the quagmire: (1) all developers engage in a job rotation scheme in customer support, (2) a small set of developers (2–3) engage in a job rotation scheme in customer support, (3) a permanent staff is appointed 2nd line customer support. Alternative (2) did not receive much interest because it was argued that none wanted the significant burden of up to 1/2 of their work in customer support. Alternative (3) lacked management approval because it would involve significant costs and because MapIT didn't have a clear idea of the volume of customer support work.

There were critical comments from some of MapIT managers to alternative (1) as well. Primarily, the concerns were related to the ability of a rotation system to ensure high quality customer support to customers in case rotation introduced additional handovers between people and extra administration. There were also concerns whether developers would accept working with support.

Ultimately, MapIT management selected to go forward with alternative (1). We then discussed the duration of each cycle. Suggestions ranged between 1 week and 3 months. The majority was in favor of 2-week cycles. This would also coincide with the 2-week iterations used in the development projects and hence simplify planning of the iterations. Nevertheless, job rotation to customer

support was defined as a trial that should have a moderate duration – until September 2008. At that point the researchers would conduct an extensive evaluation and a decision to continue or abandon job rotation should be made. Additionally, the trial could not commence before a designated support room was fully furnished.

4.3. Action taking

After participating in the diagnosing and action planning meetings, MapIT moved swiftly forward with the trial. In rotation, two developers, one from the Java group and one from the .NET group, would collaborate with Carl (S), sitting together in the support room. The duration of each rotation would be two weeks. We had limited contact with MapIT during the initiation of the intervention.

The reorganization involved a set of changes. Most notably, the support function became a very visible physical construction in MapIT's premises – a new support room was set up to house four desks facing each other, in a diamond shape. Each desk was equipped with two large monitors and a powerful PC. (Fig. 2 illustrates the layout.) The room had the size of two ordinary offices and had large windows toward the other office areas to improve the visual connection with the other employees. However, the room also had a door that could be closed in case the customer support team wanted fewer disturbances from the outside office area where people frequently engaged in conversation.

A rotation schedule was defined and distributed within MapIT that determined when individual developers should participate in the support work. Furthermore, Both Carl (S) and all developers were asked to register the time they spent on customer support related work. Specific forms were made available for this purpose.

Initially, the rotation schedule had intervals of two weeks to coincide with the Scrum iterations. In week six, just after our first review meeting, it was reduced to one week to accommodate dismay among developers of excessive project delays caused by resources allocated to the rotation schedule. Also, initially the rotation schedule included one developer from the Java group and one developer from the .NET group to participate in the customer support function simultaneously. In week six, this was changed to only one developer, from either the Java or .NET group. The rationale for the latter decision, we learned later, was to reduce cost.

A few weeks after our concluding interviews early September 2008, job rotation was abandoned and one of the trial participants was appointed permanent 2nd line customer support.

4.4. Evaluating

In the 5th week of the trial we conducted a preliminary review meeting for all developers and a number of managers. The developers considered the new scheme of job rotation in customer support as problematic. There was extensive 'personal code ownership' and 'esoteric ways of solving problems' that made overlapping work difficult. Next, many developers raised concern that they perceived a pressure to continue in a development role while being on 2nd line customer support. Also, there were concerns that some developers had been able to shirk the shift and threatening fairness of the rotation.

However, there were also developers that saw benefits and argued that the trial should continue for a longer period before formal evaluation. One noted that such learning initiatives take a long time before giving visible effects. Besides, all developers had positive experiences with the trial as well. These experiences were related to a positive effect on the quagmire problem, that it was valuable to be co-located with 1st line customer support for

resolving enquiries and that the trial itself, with the increased attention to the customer support problem and the dedicated customer support room, was regarded as a positive initiative from management.

We sent a report summarizing the review meeting to the participants plus the relevant managers the same day. We were then invited to a new meeting with MapIT where we learned that MapIT had decided to continue the trial for 13 more weeks. A schedule for concluding interviews and presentation of evaluation results was defined. One week after concluding the trial we conducted our final interviews. Shortly after, we made a quick summary of the results and presented them to MapIT. The comments we received were used in our in-depth evaluation.

4.4.1. Learning from rotation to customer support

Our data suggested that customer support is cognitively highly demanding work. A general lack of design and code documentation across the product range emphasized these demands. Extracting knowledge embedded in the products had to come from careful studies of the code. Alternatively, the developer on rotation could talk with the product specialist, but they did so only reluctantly. Many of the participants referred to a mild 'peer pressure' to maintain the shielding effect.

The rotation schedule discriminated participants that were unable to quickly understand and resolve unfamiliar problems. In the majority of cases, participants on rotation would deal with customer enquiries regarding products for which they had no prior experience. While some of the participants expressed no dissatisfaction with this, others found it highly difficult to *understand unfamiliar code* and subsequently resolve the customer's enquiry. Most of the support enquiries regarded .NET applications, which according to participants had simpler architectures and designs than the Java applications. Nevertheless, our data showed that Java developers struggled more than .NET developers. In particular, two of the Java developers expressed strongly that the dealing with problems in .NET applications was difficult. The other three Java programmers and the .NET developers described the tasks as easy or moderately difficult.

Furthermore, customer support work had particular characteristics. These are important contextual factors when interpreting the results. Among some participants, customer support work was perceived as *stressful and lacking freedom and creativity*. One developer complained that customer support work involved "the dealing with problems you didn't create yourself." This can be interpreted as an aspect of the specialist culture in MapIT; developers accept personal responsibility for problems with their code and simultaneously reject responsibility for other developers' code.

Adding to this, the rate of incoming customer enquiries was periodically high. Andrew (J) described customer support work as "working with a constant queue of tasks." During his two shifts he had just been able to resolve tasks at the rate they arrived – he was not able to 'clear the queue' as he had wanted. Furthermore, customer support work has strong timeliness properties; when customers have problems they make an enquiry. Simon (J&N) said:

I perceived it as a stressful situation to be in – I was sitting there and the customer expected me to already know the answer.

During the last year, Simon has shifted focus from .NET development to Java. So despite his familiarity with the .NET programming environment and some of the .NET products, he was uncomfortable with the timeliness expectations. We were also interested to know whether customers with enquiries were angry or upset but this was not mentioned. Matthew (N) explained:

It's not really a problem with frustrated customers; it's the fact that the support work destroys your rhythm. But there's also the added

tension you know – we're frequently fixing things in live, operational systems where mistakes can be fatal.

In summary, customer support work can be seen as an extreme form of customer driven development where the customers are continuously delivering requests for the tasks to be performed.

4.4.2. Institutionalization of customer support using job rotation

The participants were positive to the formalization of the support function via the support room, the explicit allocation of resources (via the rotation schedule), the co-location with Carl (S), and what they perceived as management's intention to improve the customer support function. They felt that the formalization had confirmed an organizational acceptance for the importance of support work.

Also, the formalization altered actual practice in dealing with customer support enquiries. All support enquiries were now received by Carl (S) and passed onto the developer on rotation. In our interviews, all participants talked about their personal experience with support enquiries. The altered practice included enquiries from MapIT's consultants as well. Eric (N), who had recently moved from the consultant group to the development group explained: "Prior to the reorganization we didn't tunnel the enquiries through customer support. Now this is how it happens."

Furthermore, the formalization was perceived as giving improvements in efficiency, quality, and responsibility with regards to the provision of customer support. Both participants and members from management expressed such opinions. Anthony (M) referred to positive feedback from a customer: "The customer felt MapIT was better at acknowledging the receipt of customer enquiries." Nick (M) explained: "I feel better assured that customers' requests are properly received and processed – that the requests are piped through a system that is continuously staffed." Gerry (N) commented: "I believe that the reorganization leads to more predictable and faster resolution of customer enquiries as compared to how it was before when individual developers were responsible." Furthermore, participants felt that the formalization brought legitimization for spending time on improving live systems. Teresa (J) said: "It is very expensive to only correct bugs half-heartedly instead of doing it properly – once and for all. Now we have allocated time to correct bugs properly." Andrew (J) said:

A positive aspect of the formalization is that customer support is planned and given resources. Customer support does take time. Putting a person there [in the customer support room] reflected that he should work with customer support enquiries. Before it was different – then we planned with full development time and customer support was extra work.

But a majority of participants believed that job rotation did not bring *sufficient authority and commitment* to the customer support role. Stuart (J), for example said: "The most important problem still remains to be addressed – namely that customer support becomes a respected role. A permanent staff would better reflect a view that customer support is something worth investing in." Chris (N) had a similar opinion: "I had hoped that developers, by being exposed to customers' problems with products, would become better at setting expectations to the products. But in practice it resembles setting expectations to your self – which might be difficult. I think we lack a function that would formally approve requirements." Carl (S) worried that job rotation would result in low commitment to the support tasks because developers had such strong ownership to their own products and therefore would prioritize work benefiting their own products. We also learned that participants were worried about fairness – many of them believed that other developers did not do their fair share of the work but rather blamed high pressure in the projects for avoiding support work.

Despite this, three participants found the rotation practice acceptable. Two others said that it was acceptable to rotate to customer support if there was another permanent support staff employed. The four participants from the Java group were against rotation altogether.

In a later group meeting at MapIT, where we presented intermediate results from the trial [30], we learned that there had been talks for a long time regarding the efficiency ideal. David (J) explained that the currently high load on developers inhibited the slack needed for knowledge redundancy. Another developer (not participating in the trial) commented that “the high load has been acknowledged as a problem for the last 10 years.” Many of the meeting participants laughed. After the laughter had silenced, Matthew (N) commented: “In conclusion, we can say that the resource situation contributed to the abandonment of job rotation to support.”

4.4.3. Shielding efficiency of rotation to customer support

The efficiency of the rotation scheme to shield the other developers from customer enquiries was significant. Support log data revealed that 132 enquiries were resolved by 2nd line customer support, i.e., developers on job rotation, while 130 enquiries had been passed onto 3rd line, i.e., developers off rotation. Hence, 50% of the enquiries not resolved by 1st line customer support, i.e., Carl (S), were handled by developers on rotation. Considering the short duration of the trial, a shielding effect of 50% shows the potential of job rotation to both improve knowledge redundancy and to absorb a significant amount of the disturbing customer enquiries.

However, participants' subjective opinions on the shielding effect varied. The majority of participants had observed moderate effects. For example, Stuart (J) explained: “I've noticed a certain reduction in the number of support enquiries.” Two participants had observed good shielding effect, such as Chris (N):

It is noticeable fewer support requests coming to me. [...] In particular in the latest weeks as we've been able to document quite a few of the recurring problems to simplify the task for the person in there [in the support room].

Individual participants' variable capability to resolve incoming customer enquiries can explain part of this diversity in opinion (cf. Section 4.4.1 above). Individual participants' were also likely to hold different expectations towards shielding.

In evaluating the shielding effect it is important to consider the longitudinal aspect of learning. Over time, participants would probably accumulate experience that would improve their proficiency in dealing with customer enquiries. Eric (N), having contributed 5 weeks in the support room, explained: “It works better and better. You know, we have only completed a few rounds still.” Nevertheless, Eric (N) believed that a permanent 2nd line support staff would be even more efficient. Chris (N) expressed a similar point of view: “If we continue like this, with rotation, then it will likely be better after a while – the longer people have been there – the more their knowledge diversity will increase. But I think it is a long road ahead.”

4.4.4. The individual and collective properties of knowledge redundancy

Introduction of knowledge redundancy practices had implications for the individuals as well as implications for the collective. Knowledge redundancy in MapIT implied that more than one individual gained experience with a particular product. MapIT's strong specialist culture, making specialization the norm and resulting in problems such as the quagmire, was therefore fundamentally challenged by the new practice. Prior to the trial the wide-ranging

product portfolio resulted in numerous individual knowledge territories and areas of responsibility. The trial divided this responsibility among a set of participants.

Primarily, participants associated knowledge redundancy with irrational prioritization and investment in learning for which they did not expect personal gains. Chris (N) suggested that peoples' motivation to learn about other products could be low:

Lately, after we changed the rotation to only include one developer at a time, there is more pressure to deal with the full variety of customer enquiries. But how it really works, when the developer on rotation gets an enquiry dealing with a product from the other group [Java or .NET] one can suspect that the enquiry is passed rather quickly onto 3rd line [customer support]. I don't know for sure, but because we sit there for rather short periods of time people might be reluctant to invest in understanding the enquiry. After all, it's a quite long duration until the next period on rotation.

Chris' (N) comment was a reminder that participants sought clear use of new knowledge. Learning without a justified goal was in conflict with performance, which surfaced as the fundamental rationality among the respondents. Stuart (J) said: “The short term memory is easily lost. When you re-enter customer support you have to learn everything anew.” Stuart and Chris' comments made us aware that participants did not expect to keep knowledge from customer support work ‘alive’ while doing their normal development duties. In a sense, the customer support knowledge was only useful in that particular context. Michael (N) had a much more positive attitude to the trial in general but commented that learning from job rotation to customer support was ‘fragmented’ although he believed that the gaps would be filled as he gained more experience.

While specialist knowledge was associated with high performance, knowledge redundancy had the opposite meaning. Teresa (J) said: “The right person would solve this problem much quicker and I didn't feel useful.” In the interview she made it clear that she felt unproductive. This opinion is reflected in another negative property of knowledge redundancy: Participants felt that the knowledge redundancy perspective to learning consumed a lot of extra time – there was a significant aspect of personal inefficiency. Participants warned about the overhead in just getting up to speed and made comparisons to “having been on vacation.”

Participants' opinions regarding collective disadvantages focused on economical implications of knowledge redundancy. It was most prominent among the Java developers. Simon (J&N) said: “Economically it is directly unprofitable. We could be much more effective on the tasks we need to do, the tasks that are already behind schedule. Instead of sitting there [in the customer support room] wasting time!” During the interview he was clearly upset. Some found knowledge integration between the two technology camps to be irrelevant. David (J) claimed: “People from the two camps [Java and .NET] don't want or need knowledge from the other camp to solve their daily tasks.” His statement succinctly summarizes how MapIT's specialist culture makes knowledge redundancy irrational; there was no good reason for seeking knowledge outside your natural territory.

The overhead of building knowledge redundancy was considered damaging to the efficiency of completing projects. Because many of MapIT's projects were small, the absence of one developer for the support cycle had visible negative impact. David (J) argued: “We're not many developers – the Java group, for example, has only five members. So when one member is doing customer support we're losing 20% of the manpower. Considering that we have far too few developers as it is, it is obvious that the rotation creates delays in the projects!” Among the .NET developers the negative opinions were much more moderate. Michael (N) said:

We have an enormous number of products and many different types of support enquiries. Thus, there will be a tendency for people to have to bother other people, in order to understand the tasks. [...] You know, I've been positive towards rotation from the start, but I think it would be more efficient with a permanent staff.

A finding that seems to illuminate the modest perception of being rewarded for collective contribution was the very strong emphasis on fairness among the participants. The difficulty of ensuring fairness was mentioned as a barrier by many of the participants. Gerry (N) saw no disadvantages except that the participation in customer support should have been fairer. We expect also the lower status of customer support work to be important in explaining the problem with implementing rotation fairly at MapIT. Many expressed the desire for more discipline among the participants in adhering to the schedule. Interviews with management confirmed the difficulty of fairness. Anthony (M) provided a very interesting view:

[The experience of participating in a rotation scheme] is much related to individuals' character. There are some which are interested in many things and thinks that it's ok to be in that [rotation] process. They don't see any problems by postponing their current tasks to participate in customer support. They accept that management takes the blame for delays and thinks diversity is attractive too. Then, on the other hand, I have people that are very committed to their projects.

We also searched our data for clues to illuminate MapIT's underlying tendency of rewarding the two learning types. Ken (C), a consultant with a significant interest in our action research due to his prior experience as a customer support manager in a different organization, perceived developers as professional only when they specialized: "Being a developer, I want to be a specialist and do my job 100% professionally." From our meetings with MapIT we got the impression that Ken (C) reflected a common opinion among the developers. This suggested that professionalism was an important driver for the specialist culture. Similarly, it would suggest that developers getting involved in a diverse set of activities risked becoming less professional.

4.4.5. Job rotation's influence on broadening minds

Our data revealed that job rotation helped to broaden participants' area of concern. Participants' experience with different products seemed to stimulate increased identification with organizational concerns such as the need to improve maintainability of the products and to engage in more strategic product portfolio planning.

From our diagnosing and other interviews we got the impression that product portfolio planning was ad hoc. In our interviews with non-participants, we asked Ken (C) whether developers' experience with the unwieldy product portfolio could contribute to a better product strategy in MapIT:

Yes, and their experiences can be important for our top management. For example, in marketing. We often say that we don't get help from marketing during our sales efforts. But how are they [marketing] supposed to understand our products when we have an ocean of them? [...] As it is today MapIT's strength is that we can do everything. But this will kill us in the end. [...] We have all possibilities to cultivate our strategy and become our country's best provider of certain products. Now we're best at providing 'whatever you want.'

Stuart (J) also expressed opinions regarding this challenge:

By working with different products you realize the product's importance to the customer. And this understanding is useful for us in the

near future as we need to make some decisions regarding what products to keep in the [product] portfolio.

Hence, the trial was likely influential in exposing participants to organizational challenges and stimulating a more active debate within MapIT as to how the company's product strategy should look like. We think this is an attribute of the personal experience people get from job rotation which triggers deeper and more vigilant engagement.

Participants mentioned situations where the practical collaboration between people from different specialties stimulated creativity and new ideas. Overlapping knowledge was identified as important to enable this. Eric (N) commented:

It is much easier to ask for help, and to receive help, if the others are familiar with your skills. From the different perspectives of different people useful ideas often come up. Similarly, you have knowledge that others might benefit from. And this knowledge is not shared unless you know what these other people are doing.

He then explained how prior experience with a GIS technology had become useful to one of the Java projects, partly due to the job rotation trial. Carl (S) commented that improving overlap between peoples' skills would benefit flexibility and subsequently reliability of the customer support function in case people should become ill.

The increasing pressure to work on a wider range of products resulted in reflection upon the challenge of improving cooperation. There was a need to share knowledge about the products. An interesting thing happened during the first week of Andrew's (J) shift in support. He saw the need to share information among the participants. He demonstrated how a tool based on Wiki technology could capture frequently used techniques to resolve customer support enquiries. Andrew (J) and Michael (N) were working together on this shift and rapidly entered available notes and techniques into the tool. Subsequently, all participants contributed and benefited from the support-Wiki. Additionally, Carl (S) claimed that the Wiki "created a common platform for dealing with product issues." Participants argued that the very low barrier to entering information in the Wiki was important in the hectic customer support environment. Thus, it seems fair to assume that the Wiki could be an important resource for sharing knowledge across the product range in the future.

5. Discussion – specifying learning

In this section, we discuss the theoretical and practical implications of our findings. In particular, we will specify how the findings contribute to learning and in answering our research question: *What benefits and challenges can arise from introducing knowledge redundancy interventions based on job rotation in software organizations?* We have grouped the discussion into four major themes: The cognitive demands of knowledge diversity, the complex rationality of knowledge redundancy, innovative learning and institutionalization of job rotation.

5.1. The cognitive demands of knowledge diversity

The evaluation demonstrated that improving knowledge redundancy by means of job rotation to customer support was challenging for the participants. The five participants belonging to the .NET group found rotation acceptable while the four participants from the Java group did not. We see several factors that can explain participants' resistance to job rotation to customer support.

First, MapIT's wide range of products reflected and enacted a strong specialist culture. The participants' had very limited knowledge about the other products before the trial. In order to improve knowledge redundancy each participant in the trial had to expand

his/her knowledge diversity. Hence, for many of the participants in the trial just the mere practice of working on 'someone else's code' was a significant change. This was particularly pronounced when developers had to cross the .NET/Java boundary. And because there was little prior tradition for collective coding, there was also limited documentation. This adds to the difficulty of altering practice.

Second, learning about fundamentally new things is more demanding than learning about things close to what we already know [44]. Some of the participants complained that it was 'beyond their capabilities' to work with a diverse set of products. Furthermore, as people engage in work they simultaneously seek to 'make sense' of the work. Because knowledge about unfamiliar products is harder to systematize it is also harder to make sense of it and see how it can be useful [33]. Hence, the degree of familiarity with the technology appeared to mitigate the cost of learning [61]. Unfamiliarity adds to the individuals' perceived cost of knowledge diversity.

Third, prior research has showed that customer support work is cognitively demanding and depends heavily on problem-solving skills as well as abilities to quickly understand the structure and implementation of the software in question [18,74]. Our evaluation showed large distribution in participants' ability to cope with the support tasks. Adding to the demands of customer support work is the randomness of enquiries that place greater demands on peoples' ability to deal with prioritization and switching between tasks. However, we received no negative comments regarding the randomness of enquiries from participants on rotation.

Combined, these three factors resulted in a very demanding situation for participants on rotation in the support room. Considering this, it is encouraging to note that five of the nine participants actually found the trial acceptable and the cognitive demands of knowledge diversity bearable. We see primarily two factors that contribute to explain this.

First, the trial did in fact make a significant contribution towards reducing the quagmire problem. As a result, developers off rotation had fewer interruptions during project work. Hence, the participants did receive personal benefits for their collective efforts in customer support. Some participants also believed that the shielding effect would increase had the job rotation practice been continued.

Secondly, many of the participants found it interesting to learn about the range of MapIT's product. They saw the trial as an opportunity to become acquainted with different products and hence became engaged in a wider set of MapIT's activities. This would probably not have occurred if participation was optional because the projects had priority. However, our evaluation suggests that it was the most experienced developers that found most interest in the rotation practice. One explanation for this can be that integrative learning is related to work experience [44]. Being able to appreciate an integrated set of concerns can imply a more profound understanding of software development, for example customers' vital role in shaping technology [41], the intertwined dependencies between developers and users [28] and various aspects concerning product portfolio management and development strategies. Correspondingly, our evaluation can imply that junior developers find specialized work challenging enough.

5.2. The complex rationality of knowledge redundancy

Our evaluation revealed that knowledge redundancy had significant negative connotations among the participants. We learned that many participants considered knowledge redundancy as irrational. To understand the irrationality of knowledge redundancy, we should first consider the context in which the trial was carried out. Rationality (and irrationality) is strongly embedded in culture and daily practice. Although the organizational culture may often

contribute to align individuals' values, a specialist organizational culture is likely to reduce collective participation [48]. Knowledge redundancy in MapIT was limited at the start of the trial due to a proliferation of small projects and very little cross-product experience among developers. Developers' tradition of working individually was not merely explained by their own preferences but also by MapIT's culture, which accepted, motivated, or even fostered specialization. We found that MapIT enacted a strong specialist culture where rationality was dominated by personal efficiency on separate projects. Similarly, the specialist culture seemed to reduce the status of customer support work. Many respondents wanted to 'get rid of' customer support work by allocating it to some other person, i.e. another specialist.

Attempts to change values and corresponding rationality will normally meet significant resistance. Argyris and Schön [5] argued that such changes require open discussion about the ideas and values that govern action. A basic tenet of action research is to enable such open discussion through explicitly problematizing existing values and rationalities by considering different alternative strategies for reaching a better situation [10]. Through the diagnosing of the action research we sought to problematize the specialist culture and the lack of collective actions that could improve knowledge redundancy and reduce the quagmire problem. Subsequently, with the trial of job rotation, MapIT legitimized and encouraged collective work that could contribute to 'boot-strap' the change.

However, while the shielding effect obtained with job rotation was significant, an important insight from the trial was the hitherto unknown complexity of the quagmire problem. Although the collective efforts of the participants contributed to contain 50% of the customer enquiries within the support room, we learned that dealing with the quagmire problem was not really vital for the Java developers. Because the Java-based products had a much lower customer base there were very few customer support enquiries regarding Java-based products. Hence, Java developers did not expect any collective benefits from the trial and this added to the perceived irrationality of 'helping' the .NET developers. Presumably, some time into the future even the Java-based products would trigger customer support enquiries, but none of the participants commented upon this. This finding suggests that the perceived usefulness of knowledge redundancy is also moderated by the expected time-span of receiving collective benefits.

Rationality of collective knowledge was demonstrated by the support Wiki however (cf. Section 4.4.5). Its collective and nearly immediate advantage for the developers on rotation contributed to its significant success in MapIT. An interesting aspect of the Wiki was that it became a tool for sharing knowledge among Java and .NET developers alike. Information in the Wiki was appropriate for the type of tasks being done in the support room and it helped the developer on rotation to resolve the customer enquiry without having to spend a long time trying to understand unfamiliar code or engaging the product specialist. As such, the Wiki constituted a highly effective tool for improving knowledge redundancy. The Wiki grew continuously during the trial and accumulated tips and problem solving stories for the range of products. Our later meetings with MapIT revealed that the Wiki had become a popular tool for 'practical documentation.'

5.3. Innovative learning

Our evaluation found support for claims that job rotation enables people to understand larger aspects of an organization's processes [20]. Participants in the trial had reflected upon various negative implications of the wide product portfolio as well as the need for better coordination between development, customer support, and marketing. The increased overlapping of knowledge contributed to cross-fertilization of new knowledge.

In addition to a wider understanding the actual experience with different roles means that people bring concrete contributions to the role. The actual experience breaks down barriers created by projects, products, and departments. Further, from actual experience it is possible to tap into the tacit knowledge that define many of an organization's routines and technologies [68]. For example, the piloting of the support Wiki happened when a participant saw a practical potential to solve a problem belonging to the role using his prior knowledge of the appropriate technology (cf. Section 4.4.5). Our evaluation brings evidence to the claim that job rotation contributes to innovative performance [46]. Bearing in mind that the trial involved rotation into a customer support role only, we expect that job rotation into a variety of roles would yield more extensive benefits.

5.4. Institutionalization of job rotation

If knowledge redundancy is to be maintained it is important that the organization has sustainable practices. Our evaluation showed that job rotation to customer support had positive effects on knowledge redundancy but that its implementation was not attractive enough to warrant continued use.

The strongest opponents to job rotation were the Java developers. They did not see any significant benefits from participating and the trial became irrational. A problem with the implementation of job rotation in the trial was the long chain of means-end relations that reduced the perceived usefulness of participating. At present, the Java-based products did not create many customer enquiries so Java developers' efforts did not bring any observable gains. This finding suggests that attempts to institutionalize job rotation should strive to accommodate shorter-term benefits for the participants. The choice to reduce staffing from 2 to 1 developer on each rotation cycle was probably contributing to Java developers' negative opinion as it increased the cognitive load on the person on rotation.

For the .NET developers job rotation to customer support brought observable benefits and was an acceptable solution, given that it was supplementing another permanently hired staff that did customer support work full time. The .NET developers argued that the role itself did not create sufficient continuity and ownership for the customer support work that had to be done. This finding suggests that job rotation is more appropriate to complement and enrich existing roles rather than defining new roles.

Also, we found technology to play a very important role in enabling collaboration and knowledge sharing between people. When technology diversity is large it is more difficult to institutionalize knowledge redundancy. Management has a large influence on accommodating knowledge redundancy from their product portfolio planning.

Another general problem with the implementation of job rotation in MapIT was that short cycles created conflicts with project work. Our evaluation revealed that participants did not consider the problem equally significant. Further, due to the modest time-span of the trial we believe that stronger peer-pressure to participate would have evolved had the trial continued. When members of a group contribute to a common goal it is normal that group norms are shaped and subsequently becomes part of the guiding power relationships [7]. There was already considerable displeasure with some participants that had omitted their turn on rotation. Also, MapIT management would have had strong motivation for establishing common guidelines for all participants that would provide increased legitimacy for participating. As our evaluation showed, it may be difficult for individuals to justify building diverse knowledge, so it is important that the organization establish such legitimacy.

If a software organization wishes to promote the building of knowledge redundancy it may need schemes for rewarding such learning. In the case of job rotation to customer support, a possible solution might be to compensate rotation participants better, ensure higher discipline in adhering to the rotation schedule and to encourage participants to share success stories leading to innovations or other benefits for the organization.

5.5. Limitations

There are four main limitations to our study. First, our investigation deals with job rotation on support work. This type of work had a negative association at MapIT. Job rotation on other types of work might not reveal the same attitudes.

Second, our intervention lasted eighteen weeks. Although we were satisfied to find an organization that allowed us to try this kind of organizational learning practice, we would have liked to see it continue longer and thus give increased insight into the sustainability of job rotation for building knowledge redundancy. Furthermore, 18 weeks is a short period of time for studying complex phenomena with clear longitudinal properties such as learning and organizational change. Thus, we must accept that our findings can only show initial effects of job rotation in terms of shielding and improved collaboration among knowledge territories.

Third, we had only limited contact with MapIT during the action taking phase of the trial. We facilitated a review meeting during this phase, but we were not in a position to significantly influence either management or participants in this period. If we were, we might have been able to convince MapIT management to continue with two developers on rotation throughout the trial. This might have given a better chance of sustained use of job rotation to customer support. The reduction from two to one developer on rotation was considered unfavorable by some of the participants. We are aware of the conflict with the collaborative spirit of some action research scholars which suggest that credibility stems solely from the workability of the tested theory [37]. We will thus have to share responsibility with MapIT for the termination of job rotation. However, very few action research project proceed exactly as planned but they can still provide substantial scientific value [22].

Fourth, the support log data were not complete. We found that one of the Java developers had not reported anything on his weekly shift as solo 2nd line customer support. Also, we found no data for week 14 of the trial. This was one of the weeks during the summer holidays and we expect the number of customer enquiries to be fairly low. In total, we do not think the incompleteness would lead to a wrong conclusion regarding the shielding effect of the trial.

6. Conclusions

We have reported upon benefits and challenges of introducing job rotation in a small software organization. Our results were obtained using action research that tried out job rotation in practice. We used existing theory of knowledge in software development and job rotation to frame and justify the research. The action research produced in-depth and comprehensive empirical data.

Job rotation can contribute to improve knowledge redundancy. Benefits of knowledge redundancy include innovation stemming from integration of different knowledge domains, and improved appreciation of organizational concerns such as coordination issues. Also, job rotation can help expose established values and rationalities.

However, a range of challenges should be considered by adopting organizations. Primarily, knowledge redundancy incurs a collective cost that must be amortized and legitimized by the organization. Secondly, job rotation can expose and confront orga-

nizational values that were previously implicit. An adoption process that accommodates open and trustful discussion among all involved people is therefore encouraged.

Acknowledgements

The work in this paper was supported by the Research Council of Norway through the project Evidence based Improvement of SOFTWARE engineering (EVIISOFT, partially funded by the Research Council of Norway under Grant No. 174390/I40). We are grateful for the constructive comments from anonymous reviewers and associate editor at Information and Software Technology. We also thank Chris Wright for valuable comments to improve clarity and readability.

References

- [1] P.S. Adler, The evolving object of software development, *Organization* 12 (2005) 401–435.
- [2] M. Alavi, D.E. Leidner, Knowledge management and knowledge management systems: conceptual foundations and research issues, *MIS Quarterly* 25 (2001) 107–136.
- [3] J.M. Allwood, W.L. Lee, The impact of job rotation on problem solving skills, *International Journal of Production Research* 42 (2004) 865–881.
- [4] J. Arent, M.H. Pedersen, J. Nørhjerg, Strategies for organizational learning in SPI, in: L. Mathiassen, J. Pries-Heje, O. Ngwenyama (Eds.), *Improving Software Organizations*, Addison-Wesley, Boston, 2002.
- [5] C. Argyris, D.A. Schön, *Organizational Learning II: Theory, Method, and Practice*, Addison-Wesley, Reading, Massachusetts, 1996.
- [6] P.G. Armour, The business of software: the organism and the mechanism of projects, *Communications of the ACM* 45 (2002) 17–20.
- [7] J.R. Barker, *The Discipline of Teamwork: Participation and Concertive Control*, SAGE Publications, Thousand Oaks, CA, 1999.
- [8] R. Baskerville, A. Dulipovici, The theoretical foundations of knowledge management, *Knowledge Management Research & Practice* 4 (2006) 83–105.
- [9] R. Baskerville, M. Myers, Special issue on action research in information systems: making IS research relevant to practice – foreword, *MIS Quarterly* 28 (2004).
- [10] R. Baskerville, T.A. Wood-Harper, Diversity in information systems action research, *European Journal of Information Systems* 7 (1998) 90–107.
- [11] F.P. Brooks, No silver bullet: essence and accidents of software engineering, *IEEE Computer* 20 (1987) 10–19.
- [12] J.S. Brown, P. Duguid, Organizational learning and communities-of-practice. Toward a unified view of working, learning, and innovation, *Organization Science* 2 (1991) 40–57.
- [13] J.S. Brown, P. Duguid, Knowledge and organization: a social-practice perspective, *Organization Science* 12 (2001) 198–213.
- [14] P.R. Carlile, A pragmatic view of knowledge and boundaries: boundary objects in new product development, *Organization Science* 13 (2002) 442–455.
- [15] P.R. Carlile, Transferring, translating, and transforming: an integrative framework for managing knowledge across boundaries, *Organization Science* 15 (2004) 555–568.
- [16] S. Chen, Task partitioning in new product development teams: a knowledge and learning perspective, *Journal of Engineering and Technology Management* 22 (2005) 291–314.
- [17] S.D.N. Cook, J.S. Brown, Bridging epistemologies: the generative dance between organizational knowledge and organizational knowing, *Organization Science* 10 (1999) 381–400.
- [18] T.A. Corbi, Program understanding: challenge for the 1990s, *IBM Systems Journal* 28 (1989) 294–306.
- [19] J. Corbin, A. Strauss, *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*, Sage, Thousand Oaks, CA, 2008.
- [20] M.M. Cosgel, T.J. Miceli, Job rotation: cost, benefits and stylized facts, *Journal of Institutional and Theoretical Economics* 155 (1999) 301–320.
- [21] A. Davies, M. Hobday, *The Business of Projects: Managing Innovation in Complex Products and Systems*, Cambridge University Press, Cambridge, UK, 2005.
- [22] R.M. Davison, M.G. Martinsons, N. Kock, Principles of canonical action research, *Information Systems Journal* 14 (2004) 65–86.
- [23] T. Dybå, Improvisation in small software organizations, *IEEE Software* 17 (2000) 82–87.
- [24] T. Dybå, A dynamic model for software engineering knowledge creation, in: A. Aurum, R. Jeffrey, C. Wohlin, M. Handzic (Eds.), *Managing Software Engineering Knowledge*, Springer-Verlag, Berlin, 2003, pp. 95–117.
- [25] A.C. Edmondson, S.E. McManus, Methodological fit in management field research, *Academy of Management Review* 32 (2007) 1155–1179.
- [26] C. Enberg, L. Lindkvist, F. Tell, Exploring the dynamics of knowledge integration – acting and interacting in project teams, *Management Learning* 37 (2006) 143–165.
- [27] T. Eriksson, J. Ortega, The adoption of job rotation: testing the theories, *Industrial & Labor Relations Review* 59 (2006) 653–666.
- [28] C. Floyd, Software development as reality construction, in: C. Floyd, H. Züllighoven, R. Budde, R. Keil-Slawik (Eds.), *Software Development and Theory Construction*, Springer-Verlag, Berlin, 1992, pp. 86–100.
- [29] B. Flyvbjerg, *Making Social Science Matter: Why Social Inquiry Fails and How It Can Succeed Again*, Cambridge University Press, Cambridge, UK, 2001.
- [30] T.E. Fægri, *Building General Knowledge in Agile Software Organizations: Experiences with Job Rotation in Customer Support*, Agile 2009, Chicago, US, 2009, pp. 46–54.
- [31] T.E. Fægri, 2010. Adoption of team estimation in a specialist organizational environment, in: XP2010 (11th International Conference on Agile Software Development), Springer-Verlag, Trondheim, Norway.
- [32] T.E. Fægri, G.K. Hanssen, Collaboration, process control and fragility in evolutionary product development, *IEEE Software* 24 (2007) 96–104.
- [33] D.C. Galunic, S. Rodan, Resource recombinations in the firm: knowledge structures and the potential for Schumpeterian innovation, *Strategic Management Journal* 19 (1998) 1193–1201.
- [34] D. Gebert, S. Boerner, E. Kearney, Cross-functionality and innovation in new product development teams: a dilemmatic structure and its consequences for the management of diversity, *European Journal of Work and Organizational Psychology* 15 (2006) 431–458.
- [35] R.L. Glass, I. Vessey, S.A. Conger, Software tasks: intellectual or clerical?, *Information & Management* 23 (1992) 183–191.
- [36] R.M. Grant, Prospering in dynamically competitive environments: organizational capability as knowledge integration, *Organization Science* 7 (1996) 375–387.
- [37] D.J. Greenwood, M. Levin, *Introduction to Action Research: Social Research for Social Change*, SAGE Publications, 2007.
- [38] O. Hauptman, K.K. Hirji, Managing integration and coordination in cross-functional teams: an international study of concurrent engineering product development, *R&D Management* 29 (1999) 179–191.
- [39] D.E. Hodgson, Project work: the legacy of bureaucratic control in the post-bureaucratic organization, *Organization* 11 (2004) 81–100.
- [40] D.G. Hoopes, S. Postrel, Shared knowledge, “glitches”, and product development performance, *Strategic Management Journal* 20 (1999) 837–865.
- [41] T.P. Hughes, The evolution of large technological systems, in: W.E. Bijker, T.P. Hughes, T.J. Pinch (Eds.), *The Social Construction of Technology Systems: New Directions in the Sociology and History of Technology*, MIT Press, Cambridge, Mass, 1987, pp. 51–82.
- [42] V.B. Kampenes, B. Anda, T. Dybå, Flexibility in research design in empirical software engineering, in: *Evaluation and Assessment in Software Engineering (EASE 2008)*, BCS, University of Bari, Italy, 2008.
- [43] R. Keil-Slawik, Artifacts in software design, in: C. Floyd, H. Züllighoven, R. Budde, R. Keil-Slawik (Eds.), *Software Development and Theory Construction*, Springer-Verlag, Berlin, 1992, pp. 168–188.
- [44] D.A. Kolb, *Experiential Learning: Experience as the Source of Learning and Development*, Prentice-Hall, New Jersey, 1984.
- [45] A. Lindbeck, D.J. Snower, Multitask learning and the reorganization of work: from Tayloristic to Holistic organization, *Journal of Labor Economics* 18 (2000) 353–376.
- [46] B.-Å. Lundvall, P. Nielsen, Knowledge management and innovative performance, *International Journal of Manpower* 28 (2007) 207–223.
- [47] K. Lyytinen, D. Robey, Learning failure in information systems development, *Information Systems Journal* 9 (1999) 85–101.
- [48] J.G. March, H.A. Simon, *Organizations*, Blackwell Publishers, Cambridge, MA, 1993.
- [49] N.B. Moe, T. Dingsøyr, Scrum and Team Effectiveness: Theory and Practice, Agile Processes in Software Engineering and Extreme Programming: 9th International Conference (XP2008), Limerick, Ireland, 2008, pp. 11–20.
- [50] N.B. Moe, T. Dingsøyr, T. Dybå, Understanding self-organizing teams in agile software development, in: *Proceedings of 19th Australian Conference on Software Engineering*, 2008, pp. 76–85.
- [51] N.B. Moe, T. Dingsøyr, T. Dybå, A teamwork model for understanding an agile team: a case study of a Scrum project, *Information and Software Technology* 52 (2010) 480–491.
- [52] G. Morgan, *Images of Organization*, Sage Publications, Thousand Oaks, CA, 2006.
- [53] S. Nerur, V. Balijepally, Theoretical reflections on agile development methodologies: the traditional goal of optimization and control is making way for learning and innovation, *Communications of the ACM* 50 (2007) 79–83.
- [54] I. Nonaka, H. Takeuchi, *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*, Oxford University Press, New York, 1995.
- [55] W.J. Orlikowski, The duality of technology: rethinking the concept of technology in organizations, *Organization Science* 3 (1992) 398–427.
- [56] W.J. Orlikowski, J. Yates, Genre repertoire: the structuring of communicative practices in organizations, *Administrative Science Quarterly* 39 (1994) 541–574.
- [57] J. Ortega, Job rotation as a learning mechanism, *Management Science* 47 (2001) 1361–1370.
- [58] B.T. Pentland, Organizing moves in software support hot lines, *Administrative Science Quarterly* 37 (1992) 527–548.
- [59] T.J. Pinch, W.E. Bijker, The social construction of facts and artifacts: or how the sociology of science and the sociology of technology might benefit each other, in: W.E. Bijker, T.P. Hughes, T.J. Pinch (Eds.), *The Social Construction of*

- Technology Systems: New Directions in the Sociology and History of Technology, MIT Press, Cambridge, Mass, pp. 17–50.
- [60] M. Polanyi, *The Tacit Dimension*, Peter Smith, Gloucester, MA, 1966.
 - [61] S. Postrel, Islands of shared knowledge: specialization and mutual understanding in problem-solving teams, *Organization Science* 13 (2002) 303–320.
 - [62] A. Rindfleisch, C. Moorman, The acquisition of utilization of information in new product alliances: a strength-of-ties perspective, *Journal of Marketing* 65 (2001) 1–18.
 - [63] P.N. Robillard, The role of knowledge in software development, *Communications of the ACM* 42 (1999) 87–92.
 - [64] D.L. Rulke, J. Galaskiewicz, Distribution of knowledge, group network structure, and group performance, *Management Science* 46 (2000) 612–625.
 - [65] K. Rönkkö, Interpretation, interaction and reality construction in software engineering: an explanatory model, *Information and Software Technology* 49 (2007) 682–693.
 - [66] M. Song, H. van der Bij, M. Weggeman, Factors for improving the level of knowledge generation in new product development, *R&D Management* 36 (2006) 173–187.
 - [67] G.I. Susman, R.D. Evered, An assessment of the scientific merits of action research, *Administrative Science Quarterly* 23 (1978) 582–603.
 - [68] K.E. Söderquist, Organizing knowledge management and dissemination in new product development: lessons from 12 global corporations, *Long Range Planning* 39 (2006) 497–523.
 - [69] A. Taylor, H.R. Greve, Superman or the fantastic four? Knowledge combination and experience in innovative teams, *Academy of Management Journal* 49 (2006) 723–740.
 - [70] F. Tell, What do organizations know? Dynamics of justification contexts in R&D activities, *Organization* 11 (2004) 443–471.
 - [71] A. Tiwana, An empirical study of the effect of knowledge integration on software development performance, *Information and Software Technology* 46 (2004) 899–906.
 - [72] C.A. Un, Managing the innovators for exploration and exploitation, *Journal of Technology Management and Innovation* 2 (2007) 4–20.
 - [73] G. von Krogh, S. Spaeth, K.R. Lakhani, Community, joining, and specialization in open source software innovation: a case study, *Research Policy* 32 (2003) 1217–1241.
 - [74] A. von Mayrhauser, A.M. Vans, Program comprehension during software maintenance and evolution, *IEEE Computer* 28 (1995) 44–55.
 - [75] D.B. Walz, J.J. Elam, B. Curtis, Inside a software-design team: knowledge acquisition, sharing and integration, *Communications of the ACM* 36 (1993) 63–77.
 - [76] X. Wang, Organizing to be Adaptive: A Complex Adaptive Systems based Framework for Software Engineering Processes, PhD thesis, School of Management, University of Bath, Bath, UK, 2007, p. 305.
 - [77] H. Welling, Four mental operations in creative cognition: the importance of abstraction, *Creativity Research Journal* 19 (2007) 163–177.